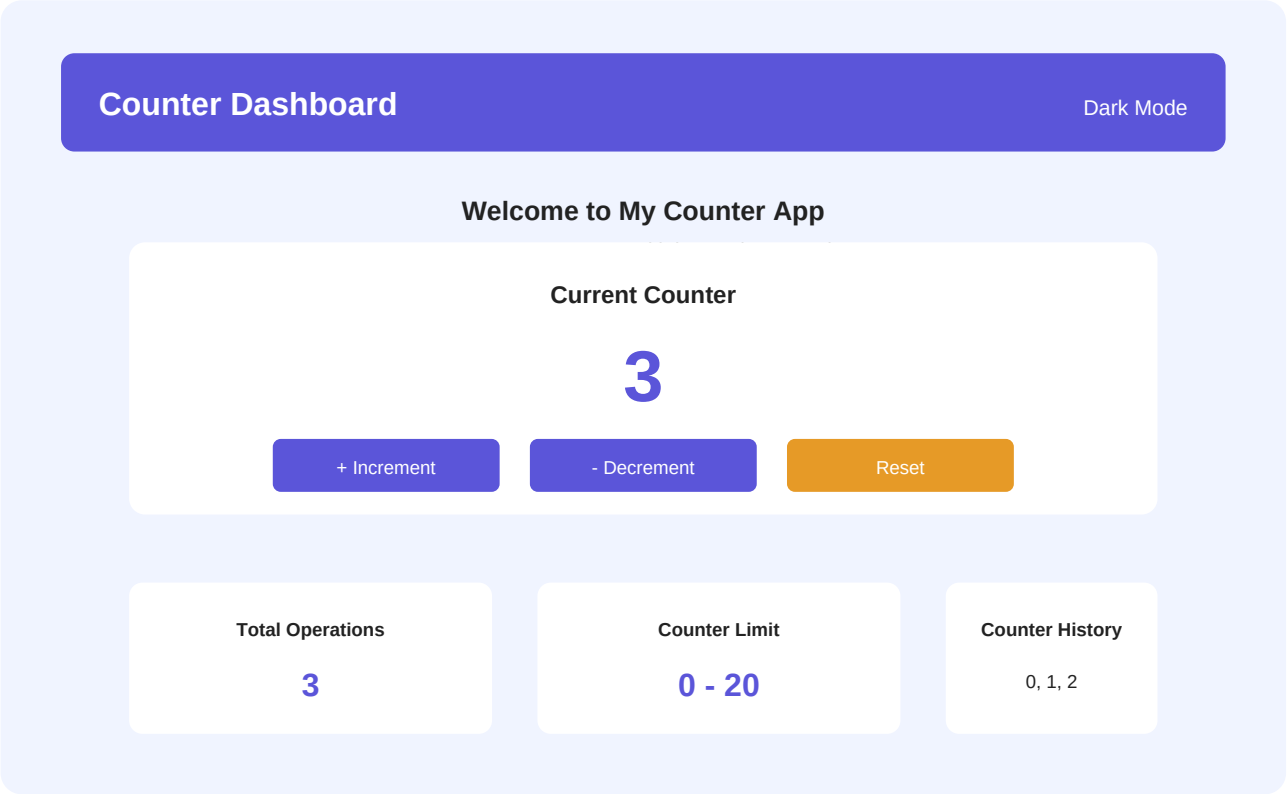
REACT COUNTER DASHBOARD

Complete Assignment: Counter Operations, Theme Toggle, Limits & History



Objective

Build a simple interactive React application using the **useState()** Hook. The dashboard combines a counter, Light/Dark Mode, counter limits, operation history and a Reset All option.

Technologies: React Functional Components, JSX, useState(), event handling, conditional rendering, JavaScript arrays and CSS.

1. Project Setup

Create a React project using Vite. Run these commands in the VS Code terminal:

```
npm create vite@latest counter-dashboard -- --template react
cd counter-dashboard
npm install
npm run dev
```

Open the project folder in VS Code. Replace the contents of **src/App.jsx** and **src/App.css** with the code below.

2. React Code: src/App.jsx

This file contains all five challenges and manages the application state with useState().

```
001  import React, { useState } from "react";
002  import "./App.css";
003
004  function App() {
005    const [count, setCount] = useState(0);
006    const [darkMode, setDarkMode] = useState(false);
007    const [history, setHistory] = useState([]);
008    const [operations, setOperations] = useState(0);
009
010    const increment = () => {
011      if (count < 20) {
012        setHistory([...history, count]);
013        setCount(count + 1);
014        setOperations(operations + 1);
015      }
016    };
017
018    const decrement = () => {
019      if (count > 0) {
020        setHistory([...history, count]);
021        setCount(count - 1);
022        setOperations(operations + 1);
023      }
024    };
025
026    const reset = () => {
027      setCount(0);
028    };
029
030    const resetAll = () => {
031      setCount(0);
032      setHistory([]);
033      setOperations(0);
034    };
035
036    return (
037      <div className={darkMode ? "app dark" : "app light"}>
038        <header>
039          <h1>Counter Dashboard</h1>
040          <button
041            className="theme-btn"
042            onClick={() => setDarkMode(!darkMode)}
043          >
044            {darkMode ? "Light Mode" : "Dark Mode"}
045          </button>
046        </header>
047
048        <main>
```

App.jsx (continued)

```
049          <h2>Welcome to My Counter App</h2>
050          <p>Manage your counter with interactive controls.</p>
051
052          <section className="counter-card">
053            <h3>Current Counter</h3>
054            <div className="count">{count}</div>
055
056            <div className="buttons">
057              <button onClick={increment} disabled={count === 20}>
058                + Increment
059              </button>
060              <button onClick={decrement} disabled={count === 0}>
061                - Decrement
062              </button>
063              <button className="reset-btn" onClick={reset}>
064                Reset
065              </button>
066            </div>
067
068            {count === 20 && (
```

```
            <p className="message">Maximum limit reached!</p>
          )}
          {count === 0 && (
            <p className="message">Minimum limit reached!</p>
          )}
        </section>

        <section className="stats">
          <div className="stat-card">
            <h3>Total Operations</h3>
            <h2>{operations}</h2>
          </div>
          <div className="stat-card">
            <h3>Counter Limit</h3>
            <h2>0 - 20</h2>
          </div>
        </section>

        <section className="history-card">
          <h3>Counter History</h3>

          {history.length === 0 ? (
            <p>No history available yet.</p>
          ) : (
            <ul>
              {history.map((value, index) => (
                <li key={index}>Previous Value: {value}</li>
              ))}
```

App.jsx (continued)

```
            </ul>
          )}

          <button
            className="clear-btn"
            onClick={() => setHistory([])}
          >
            Clear History
          </button>
        </section>

        <button className="reset-all" onClick={resetAll}>
          Reset All
        </button>
      </main>

      <footer>
        <p>React JS | useState Hook | Interactive Dashboard</p>
      </footer>
    </div>
  );
}

export default App;
```

3. CSS Code: src/App.css

The CSS controls the layout, cards, buttons, responsive design and both themes.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

.app {
  min-height: 100vh;
  font-family: Arial, sans-serif;
  text-align: center;
  transition: 0.3s;
  padding: 25px;
}

.light {
  background: #f0f4ff;
  color: #222;
}

.dark {
  background: #151827;
  color: white;
}

header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  background: #5b55d9;
  color: white;
  border-radius: 12px;
}

button {
  padding: 12px 20px;
  margin: 8px;
  border: none;
  border-radius: 8px;
  background: #5b55d9;
  color: white;
  cursor: pointer;
  font-size: 15px;
}

button:hover {
  opacity: 0.85;
}
```

App.css (continued)

```

button:disabled {
  background: #999;
  cursor: not-allowed;
  opacity: 0.6;
}

.theme-btn {
  background: #252b45;
}

main {
  max-width: 750px;
  margin: 35px auto;
}

main > p {
  margin: 12px 0 25px;
}

.counter-card,
.stat-card,
.history-card {
  padding: 25px;
  margin: 20px 0;
  border-radius: 15px;
  background: white;
  color: #222;
  box-shadow: 0 4px 15px #00000015;
}

.dark .counter-card,
.dark .stat-card,
```

```
.dark .history-card {
  background: #252b45;
  color: white;
}

.count {
  font-size: 65px;
  font-weight: bold;
  color: #5b55d9;
  margin: 20px;
}

.reset-btn {
  background: #e69a27;
}
```

App.css (continued)

```

.stats {
  display: flex;
  gap: 20px;
}

.stat-card {
  flex: 1;
}

.stat-card h2 {
  margin-top: 15px;
  color: #5b55d9;
}

.history-card ul {
  list-style: none;
  margin: 15px;
}

.history-card li {
  padding: 10px;
  border-bottom: 1px solid #ccc;
}

.message {
  color: #e04444;
  font-weight: bold;
  margin-top: 15px;
}

.clear-btn {
  background: #dc5252;
}

.reset-all {
  background: #222;
  width: 180px;
}

footer {
  margin-top: 35px;
  padding: 15px;
}

@media (max-width: 600px) {
  header {
    flex-direction: column;
```

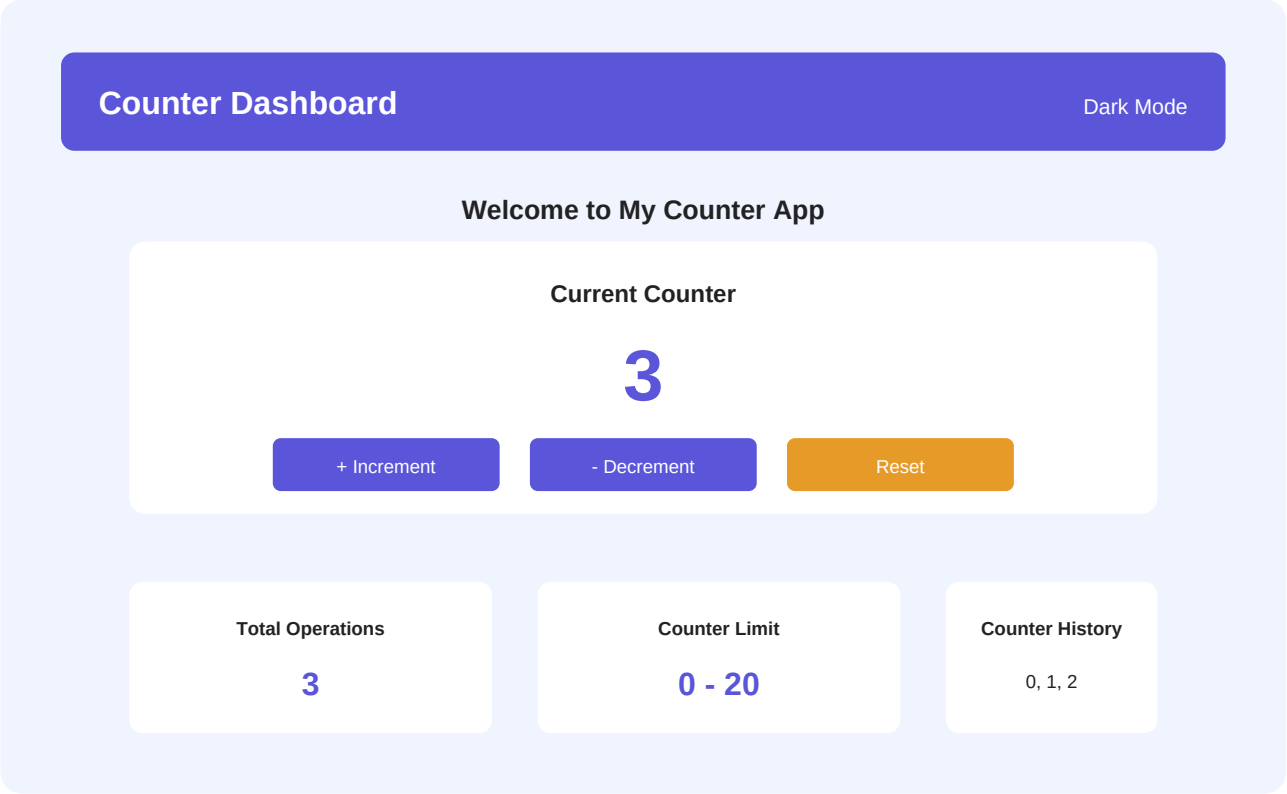
App.css (continued)

```
    gap: 15px;
  }

  .stats {
    flex-direction: column;
    gap: 0;
  }

  .buttons button {
    margin: 5px;
  }
}
```

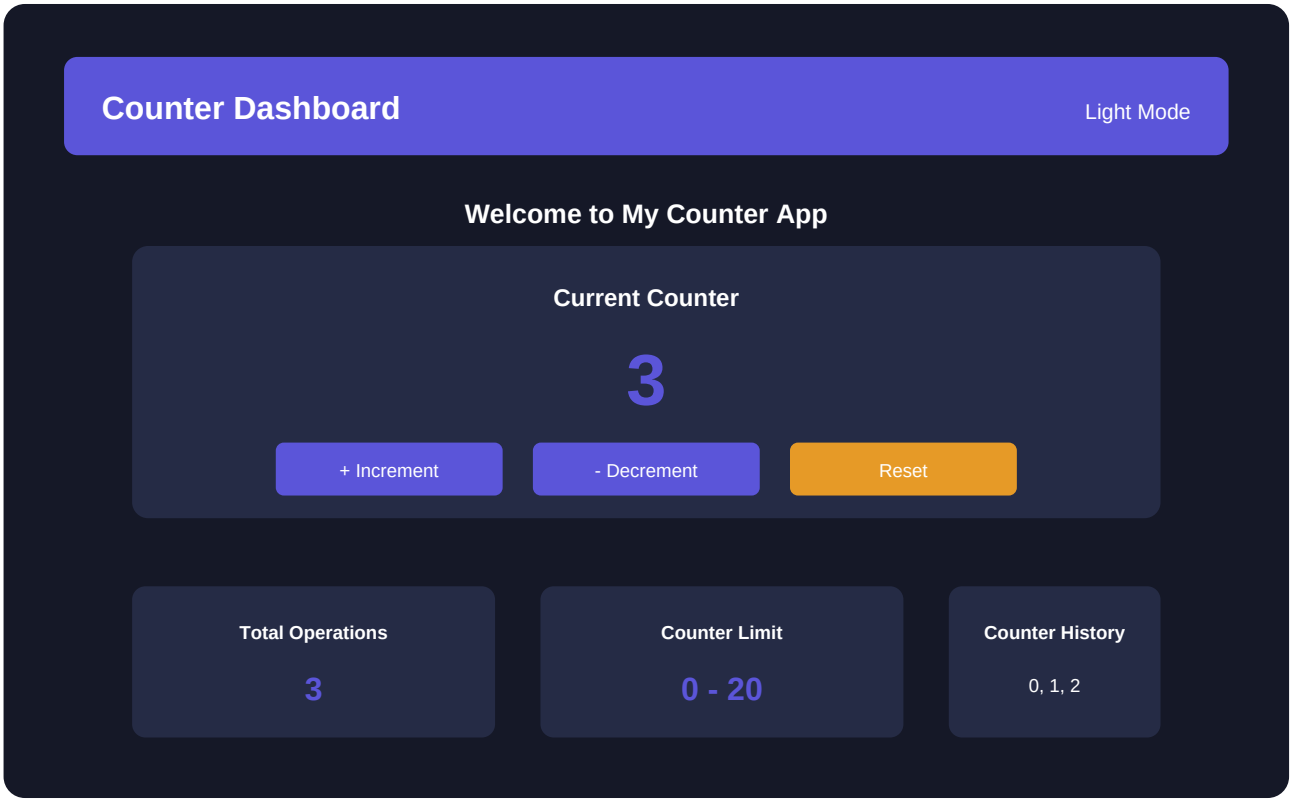
4. Expected Output

The following is an illustrative view of the dashboard. The actual counter value and history change as the user clicks the buttons.

Light Mode



Dark Mode



Sample interaction

Starting value: 0. Click Increment three times. The current value becomes 3, the total operations becomes 3, and the history shows the previous values 0, 1 and 2. Clicking Decrement records the value before decreasing it.

5. Challenge Checklist

Challenge	Implemented functionality
01: Counter Operations	Initial value 0, Increment, Decrement and Reset buttons.
02: Theme Toggle	Switches between Light Mode and Dark Mode without refreshing.
03: Counter Limits	Minimum 0 and maximum 20; buttons disable at the limits and a message appears.
04: History Tracker	Stores previous values, displays history, clears history and counts operations.
05: Interactive Dashboard	Combines all features and includes Reset All.

6. Important Notes

• The History list stores the counter value **before** each increment or decrement.
• The Reset button changes the counter value to 0 but keeps the history and operation count.
• Clear History removes only the history entries.
• Reset All sets the counter to 0 and clears both history and operation count. It does not change the selected theme.

7. Conclusion

This React application demonstrates how the useState() Hook can manage multiple pieces of state and update the user interface dynamically. It also demonstrates event handling, conditional rendering, array methods and dynamic CSS styling.

How to run: Save both files, run `npm run dev` in the project terminal, and open the localhost address shown by Vite.